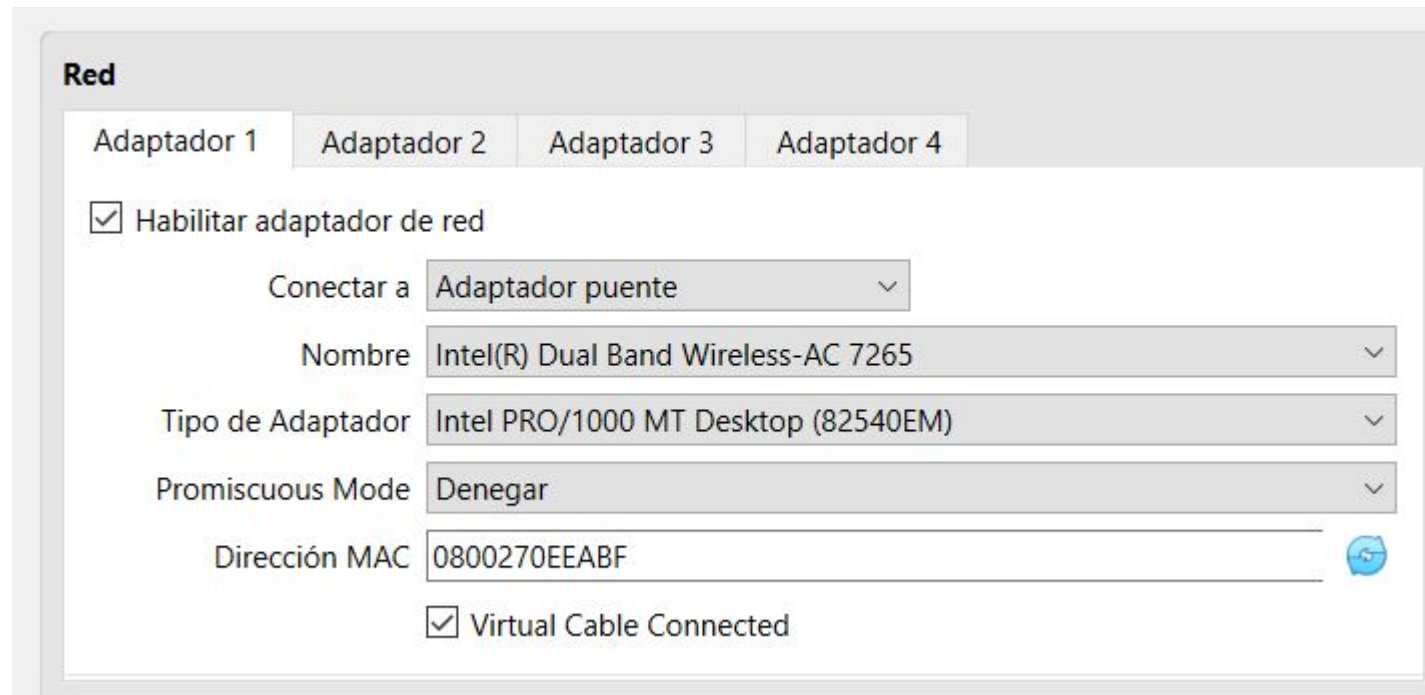


Criando de um servidor web em MV

MSC. JOSE ALFREDO MENDOZA PEÑALOZA

Conexão de rede MV

Como primeiro passo, conectaremos a máquina virtual à rede da internet a partir do nosso computador, nesse caso usaremos a configuração Bridged Adapter (Adaptador em ponte) e nosso adaptador de rede; através pela configuração de rede da máquina virtual.

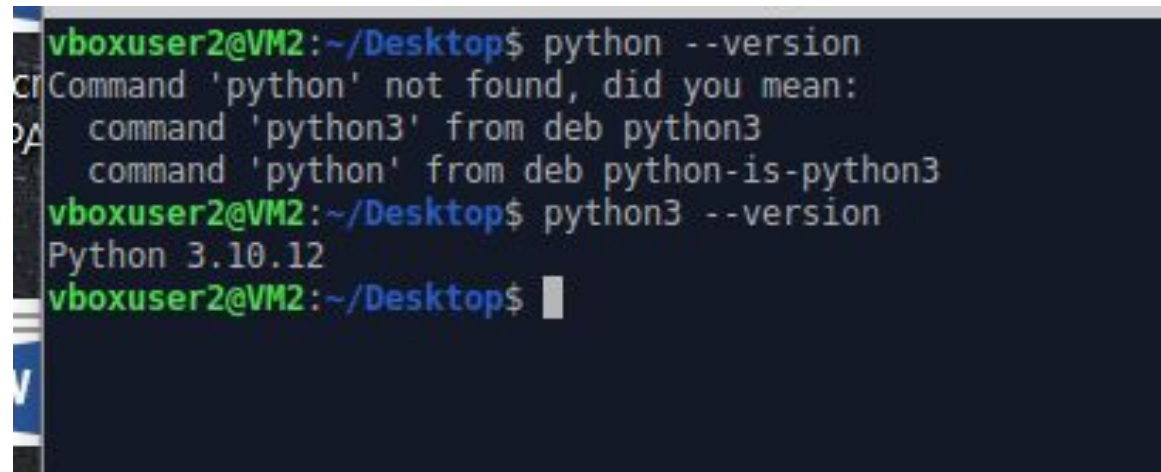


The image shows a screenshot of a virtual machine's network configuration window. The window has a title bar labeled "Red" and four tabs: "Adaptador 1", "Adaptador 2", "Adaptador 3", and "Adaptador 4". The "Adaptador 1" tab is selected. Inside the tab, there is a checkbox labeled "Habilitar adaptador de red" which is checked. Below this, there are several configuration options: "Conectar a" is set to "Adaptador puente" (Bridged Adapter); "Nombre" is set to "Intel(R) Dual Band Wireless-AC 7265"; "Tipo de Adaptador" is set to "Intel PRO/1000 MT Desktop (82540EM)"; "Promiscuous Mode" is set to "Denegar" (Deny); and "Dirección MAC" is set to "0800270EEABF". At the bottom, there is another checkbox labeled "Virtual Cable Connected" which is also checked. A small blue icon is visible next to the MAC address field.

Instalação do Python

O próximo passo é instalar o Python em nossa máquina virtual, usando os seguintes comandos.

```
sudo apt update  
sudo apt install python3 python3-pip -y
```



```
vboxuser2@VM2:~/Desktop$ python --version  
Command 'python' not found, did you mean:  
  command 'python3' from deb python3  
  command 'python' from deb python-is-python3  
vboxuser2@VM2:~/Desktop$ python3 --version  
Python 3.10.12  
vboxuser2@VM2:~/Desktop$
```

Diretório do servidores web

O próximo passo é criar uma pasta onde armazenaremos nosso servidor web, usando os seguintes comandos.

```
mkdir servidor_flask
```

E entraremos na pasta criada.

```
cd servidor_flask
```

```
vboxuser2@VM2:~/Desktop$ mkdir servidor_flask
vboxuser2@VM2:~/Desktop$ cd servidor_flask
vboxuser2@VM2:~/Desktop/servidor_flask$ pwd
/home/vboxuser2/Desktop/servidor_flask
vboxuser2@VM2:~/Desktop/servidor_flask$ s
```

Ambiente virtual venv

O próximo passo é a criação de um ambiente virtual (venv) instalado por meio do Python. Ambientes virtuais são usados para a criação e modificação de versões do servidor e do Python sem alterar o sistema geral.

Código de instalação do venv

```
sudo apt update  
sudo apt install python3.10-venv
```

criação de um ambiente virtual

```
python3 -m venv venv
```

```
servidor_flask/  
|  
└─ venv/
```

Inicialização do ambiente virtual

Após a instalação e criação do ambiente virtual, iremos iniciá-lo utilizando os seguintes comandos.

```
source venv/bin/activate
```

Ao iniciar o venv, o terminal exibirá que estamos trabalhando em um ambiente virtual, mostrando a sigla venv no início do comando que estamos usando.

```
vboxuser2@VM2:~/Desktop/servidor_flask$ rm -rf venv
vboxuser2@VM2:~/Desktop/servidor_flask$ python3 -m venv venv
vboxuser2@VM2:~/Desktop/servidor_flask$ source venv/bin/activate
source: command not found
vboxuser2@VM2:~/Desktop/servidor_flask$ source venv/bin/activate
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$
```

Para sair do ambiente virtual, usamos o seguinte comando.

```
deactivate
```

```
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$ deactivate
vboxuser2@VM2:~/Desktop/servidor_flask$
```

Gerenciador de servidor web (Flask)

O próximo passo é a instalação de um gerenciador de servidor web; neste caso, utilizaremos o plugin Flask oferecido pelo Python.

O comando de instalação é o seguinte:

```
pip install flask
```

Com o seguinte comando, podemos ver a versão do Flask que estamos usando.

```
pip show flask
```

```
jinja2=3.1.0 markupsafe=3.0.1 werkzeug=3.1.0
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$ pip show flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License:
Location: /home/vboxuser2/Desktop/servidor_flask/venv/lib/python3.10/site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$
```

Criação do servidor web

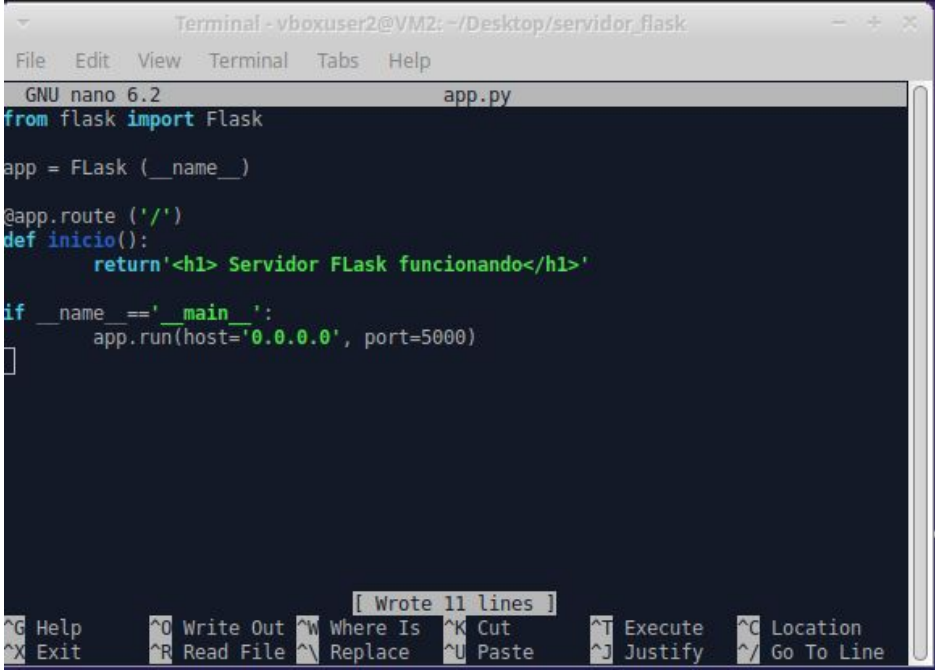
O próximo passo é a criação da aplicação em Python para criar o servidor web e seu conteúdo, onde poderemos desenvolver o design da página web desejada.

Utilizamos o seguinte comando para abrir o script GNU nano em Python e criar o código app.py.

```
nano app.py
```

Usamos o seguinte link para acessar ao servidor web local.

```
http://localhost:5000
```



```
Terminal - vboxuser2@VM2: ~/Desktop/servidor_flask
GNU nano 6.2 app.py
from flask import Flask
app = Flask(__name__)
@app.route('/')
def inicio():
    return '<h1> Servidor FLask funcionando</h1>'
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```


Criação do servidor web

Utilizamos o seguinte código para criar o servidor web.

```
GNU nano 6.2 app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def inicio():
    return '<h1> Servidor FLask funcionando</h1>'

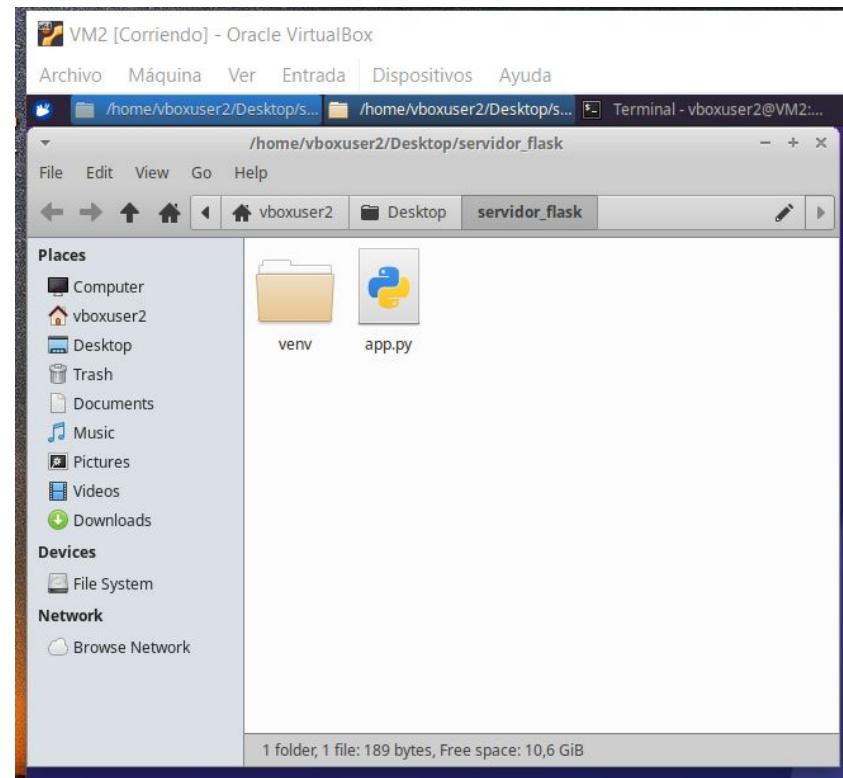
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Usamos o seguinte link para acessar ao servidor web local.

```
http://localhost:5000
```

Diretório de servidores web

Em seguida, vamos para a área de trabalho e procuramos a pasta do projeto do servidor web para revisar seu conteúdo, a pasta venv e o aplicativo Python app.py.

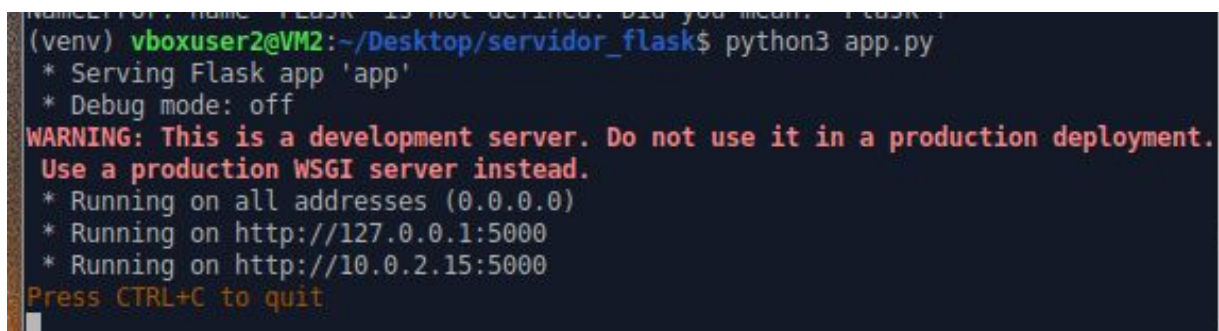


Inicialização do servidor web

Em seguida, configuraremos o servidor usando Python com os seguintes comandos.

```
python3 app.py
```

Após a execução do comando, o terminal exibirá algumas informações sobre o nosso servidor web, como endereço, velocidade, conexão, etc.

A screenshot of a terminal window with a dark background. The prompt is '(venv) vboxuser2@VM2:~/Desktop/servidor_flask\$'. The command 'python3 app.py' has been executed. The output shows: '* Serving Flask app \'app\'', '* Debug mode: off', a red warning message 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.', and three lines indicating the server is running on 'all addresses (0.0.0.0)', 'http://127.0.0.1:5000', and 'http://10.0.2.15:5000'. The prompt 'Press CTRL+C to quit' is at the bottom.

```
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
```

O link para acessar nosso servidor web é “http://localhost:5000”

```
http://localhost:5000
```


Acesso ao servidor web local em outros dispositivos

Para acessar o servidor web local a partir de outros dispositivos, primeiro precisamos saber o endereço IP do nosso servidor na rede local. Para isso, usamos o seguinte comando.

```
hostname -I
```

O endereço da nossa rede local à qual o nosso servidor web se conecta aparece abaixo, neste caso "192.168.0.249".

```
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$ hostname -I
192.168.0.249 2804:14c:65e4:832e:a1d3:6a70:bff4:90bc 2804:14c:65e4:832e:44:8480:
8dbc:ae95 2804:14c:65e4:832e:574e:7b3:d978:145d
(venv) vboxuser2@VM2:~/Desktop/servidor_flask$
```

Agora, para acessar nosso servidor local, usaremos o mesmo endereço IP, adicionando apenas o gateway de conexão definido para o acesso em nossa aplicação Python, neste caso, "5000". Em seguida, para acessar o servidor local em outros dispositivos, usaremos o link "http://192.168.0.249:5000".

Abas do servidor web

Por meio do aplicativo Python (app.py), podemos criar mais abas em nosso servidor web para apresentar conteúdos diferentes.

através do seguinte código

```
@app.route('/sobre')
def sobre():
    return '<h1> Informacoes Sobre o Sistema</h1>'

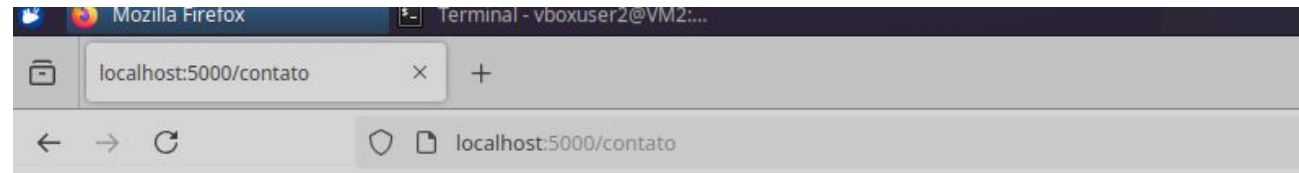
@app.route('/contato')
def contato():
    return '<h1> Pagina de contato</h1>'
```

Dessa forma, novos links serão criados, como por exemplo:

```
http://localhost:5000/
http://localhost:5000/sobre
http://localhost:5000/contato
```

Servidor web

As abas podem ser acessadas digitando os nomes descritos na aplicação de python através do navegador da web ou criando botões em nossa página da web para acesso direto.



Pagina de contato

Podemos desativar nosso servidor web pelo terminal pressionando as teclas Ctrl + C.

```
27.0.0.1 - - [02/Jun/2026 03:21:00] "GET / HTTP/1.1" 200 -  
27.0.0.1 - - [02/Jun/2026 03:21:00] "GET /favicon.ico HTTP/1.1" 404 -  
C(venv) vboxuser2@VM2:~/Desktop/servidor_flask$
```


Design profissional de servidor web

A partir daí, você pode criar servidores web mais complexos com um design mais profissional, criando arquivos HTML, CSS e JavaScript, usando novas tecnologias para implantar servidores e proporcionar uma apresentação mais profissional, (ex: Nginx, Apache HTTP Server, Django, MongoDB , etc).

Abaixo, um exemplo de uma aplicação Python com um design em HTML.

```
<!DOCTYPE html>
<html>
<head>
    <title>Servidor Flask</title>
</head>
<body>
    <h1>Bem-vindo ao Servidor Flask</h1>
</body>
</html>
```

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def inicio():
    return render_template('index.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0')
```


Design profissional de servidor web

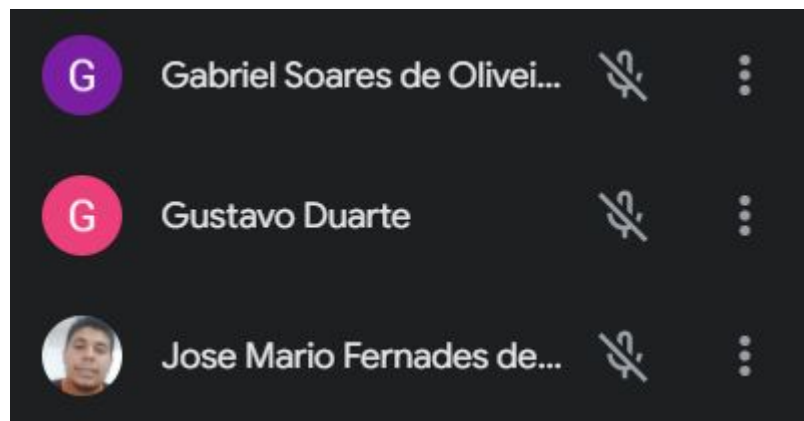
Abaixo, é apresentado um exemplo de um projeto profissional de gerenciamento de servidor web, que inclui a pasta do ambiente virtual (venv), o aplicativo Python, uma pasta de modelos para as diferentes abas do servidor web e uma pasta estática contendo os arquivos de design visual, interatividade e imagens utilizadas no servidor web.

```
servidor_flask/  
|  
├─ app.py  
|  
├─ templates/  
|   ├─ index.html  
|   ├─ sobre.html  
|   └─ contato.html  
|  
├─ static/  
|   ├─ css/  
|   ├─ js/  
|   └─ imagens/  
|  
└─ venv/
```

Fluxo de comunicação HTTP

Por fim, um resumo de como funciona a estrutura de comunicação do nosso servidor web com a internet: ao conectar-se ao navegador do cliente, ele envia uma requisição HTTP para o servidor Flask, que por sua vez a envia para a aplicação Python. A aplicação, então, recupera as informações dos templates do servidor web e envia uma resposta HTTP com as informações do website para o navegador do usuário.





Apagar directorio venv incompleto se existir

```
rm -rf venv
```

```
nano templates/index.html
```